

JavaScript 中 this 指向的全面解析

白榆_ 2025-06-20 👁 660 ⌚ 阅读6分钟

关注

JavaScript 中 this 指向的全面解析

```
13 function Vehicle() {  
14     this.a = 2;  
15     console.log(this);  
16 }  
17 var obj = new Vehicle();  
18 // this指向Vehicle这个new出来的对象obj
```

在 JavaScript 的世界里，this关键字犹如一把双刃剑，既强大又充满挑战。它的指向问题常常让开发者们困惑不已，却又在实际编程中起着举足轻重的作用。无论是构建复杂的面向对象应用，还是处理简单的函数逻辑，正确理解this的指向都是写出高效、健壮代码的关键。本文将深入探讨 JavaScript 中this指向的奥秘，通过丰富的示例和清晰的解释，帮助你彻底掌握这一核心概念。

一、全局环境下的 this 指向

在全局执行环境中，也就是在任何函数外部，this指向全局对象。在浏览器环境下，这个全局对象就是window；而在 Node.js 环境中，则是global。例如：

▼ javascript

 体验AI代码助手 

复制代码

```
1 console.log(this === window); // 在浏览器中输出true  
2 console.log(this === global); // 在Node.js中输出true
```

在全局作用域中定义的变量和函数，实际上都是全局对象的属性和方法。例如：

▼ javascript

 体验AI代码助手 

复制代码

```
1 var globalVariable = 'I am global';
2 function globalFunction() {
3     console.log('I am a global function');
4 }
5 console.log(window.globalVariable); // 输出'I am global'
6 window.globalFunction(); // 输出'I am a global function'
```

这里定义的globalVariable和globalFunction，都可以通过window对象来访问，因为在全局环境下，this就是window，它们本质上是window的属性和方法。

二、函数调用中的 this 指向

（一）普通函数调用

1. **非严格模式**：在非严格模式下，当一个函数被直接调用时（不是作为对象的方法调用，也不是通过new关键字调用等特殊方式），this指向全局对象。例如：

▼ scss

 体验AI代码助手 

复制代码

```
1 function normalFunction() {
2     console.log(this);
3 }
4 normalFunction(); // 在浏览器中输出window对象
```

2. **严格模式**：在严格模式下（在函数顶部添加'use strict';声明），普通函数中的this会指向undefined。这一变化有助于避免在函数内部意外地修改全局对象，提高代码的安全性和可维护性。例如：

▼ scss

 体验AI代码助手 

复制代码

```
1 function strictModeFunction() {
2     'use strict';
3     console.log(this);
4 }
5 strictModeFunction(); // 输出undefined
```

（二）对象方法调用

当函数作为对象的方法被调用时，`this`指向调用该方法的对象。这是`this`在面向对象编程中最常见的用法之一。例如：

▼ javascript

 体验AI代码助手 

复制代码

```
1  const myObject = {
2    name: 'John',
3    sayHello: function() {
4      console.log('Hello, my name is'+ this.name);
5    }
6  };
7  myObject.sayHello(); // 输出'Hello, my name is John'
```

在这个例子中，`sayHello`函数是`myObject`的方法，当`myObject.sayHello()`被调用时，`this`指向`myObject`，所以能够正确访问`myObject`的`name`属性。

（三）构造函数调用

当函数通过`new`关键字被调用时，它就成为了构造函数，`this`指向新创建的对象实例。构造函数用于创建对象的实例，并在实例化过程中初始化对象的属性和方法。例如：

▼ javascript

 体验AI代码助手 

复制代码

```
1  function Person(name) {
2    this.name = name;
3    this.sayHello = function() {
4      console.log('Hello, my name is'+ this.name);
5    };
6  }
7  const person1 = new Person('Jane');
8  person1.sayHello(); // 输出'Hello, my name is Jane'
```

在`Person`构造函数内部，`this`指向新创建的`person1`对象，通过`this`可以为`person1`对象添加`name`属性和`sayHello`方法。

三、事件处理中的 this 指向

（一）DOM 事件处理

在 HTML 中，当为 DOM 元素添加事件处理函数时，this 指向触发该事件的 DOM 元素。这使得我们可以方便地在事件处理函数中访问和操作触发事件的元素。例如：

ini

 体验AI代码助手 

复制代码

```
1 <button id="myButton">Click me</button>
2 <script>
3     const myButton = document.getElementById('myButton');
4     myButton.onclick = function() {
5         this.style.backgroundColor = 'blue';
6     };
7 </script>
```

当按钮被点击时，事件处理函数中的 this 指向按钮元素本身，因此可以直接修改按钮的背景颜色。

(二) 定时器回调

1. **非严格模式**：当把函数作为定时器（如 `setTimeout` 或 `setInterval`）的回调函数时，在非严格模式下，this 指向全局对象。例如：

javascript

 体验AI代码助手 

复制代码

```
1 setTimeout(function() {
2     console.log(this === window); // 输出true
3 }, 1000);
```

2. **严格模式**：在严格模式下，定时器回调函数中的 this 指向 undefined。例如：

javascript

 体验AI代码助手 

复制代码

```
1 function strictModeTimeout() {
2     'use strict';
3     setTimeout(function() {
4         console.log(this); // 输出undefined
5     }, 1000);
6 }
7 strictModeTimeout();
```

(三) 数组方法回调

对于数组的一些方法（如forEach、map、filter等），当传递回调函数时，this通常指向undefined（在非严格模式下，某些浏览器可能指向全局对象，存在兼容性问题）。不过，我们可以通过bind、call或apply方法来改变this的指向。例如：

▼ javascript

 体验AI代码助手 

复制代码

```
1  const numbers = [1, 2, 3];
2  const myObject = {
3    sum: 0,
4    addNumbers: function() {
5      numbers.forEach(function(number) {
6        this.sum += number; // 这里的this指向undefined，无法正确累加
7      }.bind(this)); // 使用bind方法将this指向myObject
8      console.log(this.sum);
9    }
10 };
11 myObject.addNumbers(); // 输出6
```

在这个例子中，通过bind(this)将回调函数中的this绑定到myObject，从而实现了正确的累加操作。

四、改变 this 指向的方法

（一）call 方法

call方法可以用来调用一个函数，并且可以指定函数内部this的指向。它接受一个或多个参数，第一个参数是this要指向的对象，后面的参数是函数本身的参数。例如：

▼ javascript

 体验AI代码助手 

复制代码

```
1  function greet(message) {
2    console.log(message + ', my name is'+ this.name);
3  }
4  const person = {
5    name: 'Bob'
6  };
7  greet.call(person, 'Hello'); // 输出'Hello, my name is Bob'
```

（二）apply 方法

apply方法与call方法类似，也是用来改变函数内部this的指向。不同的是，apply接受两个参数，第一个参数是this要指向的对象，第二个参数是一个包含函数参数的数组。例如：

ini

 体验AI代码助手 

复制代码

```
1 function sum(a, b) {  
2     return a + b;  
3 }  
4 const numbers = [3, 5];  
5 const result = sum.apply(null, numbers);  
6 console.log(result); // 输出8
```

这里null表示sum函数内部的this指向全局对象（在非严格模式下），如果在严格模式下，this将指向undefined。

(三) bind 方法

与call和apply不同的是，bind不会立即执行函数，而是返回一个绑定了this指向的新函数，可以在需要的时候调用。例如：

ini

 体验AI代码助手 

复制代码

```
1 function multiply(a, b) {  
2     return a * b;  
3 }  
4 const boundMultiply = multiply.bind(null, 4);  
5 const result = boundMultiply(5);  
6 console.log(result); // 输出20
```

这里bind(null, 4)创建了一个新函数boundMultiply，该函数内部的this指向null（在非严格模式下，实际指向全局对象），并且第一个参数固定为4，调用boundMultiply(5)时，相当于调用multiply(4, 5)。

五、箭头函数中的 this 指向

箭头函数的this指向是根据定义时所在的作用域决定的，而不是根据调用方式决定的。箭头函数不绑定自己的this，而是继承自外层作用域。例如：

javascript

 体验AI代码助手 

复制代码

```
1  const myObject = {
2    name: 'Alice',
3    arrowFunction: function() {
4      return () => {
5        console.log('Hello, my name is'+ this.name);
6      };
7    }
8  };
9  const innerFunction = myObject.arrowFunction();
10 innerFunction(); // 输出'Hello, my name is Alice'
```

在这个例子中，箭头函数继承了外层arrowFunction方法中的this指向，也就是myObject，所以能够正确输出myObject的name属性。需要注意的是，箭头函数没有自己的this，它的this指向是在定义时就确定的，不会随着调用方式的改变而改变。

六、总结

this的指向在 JavaScript 中是一个复杂但又非常重要的概念。它的指向会根据函数的调用方式、是否处于严格模式、是否作为对象方法或构造函数等多种因素而变化。在实际开发中，需要仔细分析函数的调用场景，并且可以利用call、apply和bind方法来灵活地控制this的指向，以满足代码的逻辑需求。同时，深入理解this的指向对于阅读和理解他人代码以及编写高效、可维护的 JavaScript 代码都具有重要的意义。希望通过本文的介绍，你能对 JavaScript 中this的指向有更清晰、更深入的理解，并在实际编程中能够熟练运用。

标签： JavaScript

评论 0



稀土掘金

[首页](#)

探索稀土掘金



[登录](#)